

École Marocaine des Sciences de l'Ingénieur

EMSI Casablanca

Filière / Département : Informatique

Rapport de Projet de Fin de Module

Module : **Deep Learning**

*Conception, implémentation, comparaison et analyse critique de modèles de
deep learning pour données tabulaires, images et séquences*

Présenté par : Bouka Wenn

Filière : Informatique

Année universitaire : 2025–2026

Table des matières

1	Introduction générale	2
2	Objectifs du projet	2
2.1	Objectifs théoriques	2
2.2	Objectifs pratiques	2
3	Méthodologie générale	2
4	Partie I — MLP et données tabulaires	3
4.1	Présentation du dataset et préparation des données	3
4.2	Fondements théoriques	3
4.3	Implémentation : nn.Sequential vs classe personnalisée	3
4.4	Stratégies d'initialisation des poids	4
4.5	Entraînement et sauvegarde	5
4.6	Résultats expérimentaux	5
4.7	Analyse critique	6
4.8	Question de synthèse — Partie I	6
5	Partie II — CNN et vision par ordinateur	8
5.1	Présentation du dataset Fashion-MNIST	8
5.2	Fondements théoriques	8
5.3	Implémentations manuelles vs PyTorch	9
5.4	Étude expérimentale (étude d'ablation)	9
5.5	Comparaison MLP vs CNN	10
5.6	Évaluation détaillée et visualisation des feature maps	11
5.7	Analyse critique	12
5.8	Question de synthèse — Partie II	12
6	Partie III — RNN, LSTM, GRU et système de traduction Seq2Seq	13
6.1	Présentation du corpus	13
6.2	Fondements théoriques	13
6.3	Architecture Seq2Seq avec mécanisme d'attention	13
6.4	Entraînement	13
6.5	Évaluation	14
6.6	Comparaison théorique RNN vs LSTM vs GRU	15
6.7	Analyse critique	15
6.8	Question de synthèse — Partie III	15
7	Discussion transversale finale	17
8	Limites générales du travail	17
9	Conclusion générale	18
10	Annexe expérimentale	18
10.1	Récapitulatif global des résultats obtenus	18
10.2	Paramètres et architectures	19
	Bibliographie	19

1 Introduction générale

Le deep learning constitue aujourd'hui l'un des piliers méthodologiques majeurs de l'intelligence artificielle moderne, en raison de sa capacité à apprendre des représentations hiérarchiques directement à partir de données brutes. Cette capacité repose néanmoins sur un principe fondamental : il n'existe pas d'architecture universelle capable de traiter de manière optimale tous les types de données. Chaque famille de modèles a été conçue pour exploiter une structure statistique particulière, qu'elle soit tabulaire, spatiale ou temporelle.

Le présent rapport rend compte d'un projet de fin de module structuré en trois parties indépendantes mais complémentaires : la classification de données tabulaires par un perceptron multicouche (MLP), la classification d'images par un réseau de neurones convolutionnel (CNN), et la modélisation de séquences textuelles par des architectures récurrentes (RNN/LSTM/GRU) couplées à un système de traduction automatique encodeur-décodeur avec mécanisme d'attention. Chaque partie est traitée comme une étude scientifique autonome, mais l'ensemble du document s'efforce de mettre en évidence les liens conceptuels entre ces trois approches.

2 Objectifs du projet

2.1 Objectifs théoriques

- Comprendre les mécanismes de rétropropagation du gradient et d'optimisation sous PyTorch.
- Maîtriser les spécificités architecturales du MLP, du CNN et des modèles récurrents.
- Saisir les fondements probabilistes de la modélisation de séquences et de la traduction automatique.
- Identifier les limites théoriques de chaque architecture au regard de la structure des données traitées.

2.2 Objectifs pratiques

- **Partie I** : implémenter un MLP selon deux approches PyTorch, comparer des stratégies d'initialisation, évaluer sur un dataset médical réel.
- **Partie II** : valider manuellement les opérations de convolution/pooling, mener une étude d'ablation architecturale, comparer MLP et CNN.
- **Partie III** : construire un système de traduction automatique neuronale anglais-français avec mécanisme d'attention, comparer RNN/LSTM/GRU.

3 Méthodologie générale

Une démarche méthodologique commune a été adoptée sur les trois parties : préparation rigoureuse des données (nettoyage, normalisation/tokenisation, séparation train/validation/test), implémentation sous PyTorch en respectant les bonnes pratiques d'ingénierie (`nn.Module`, gestion du device, sauvegarde via `state_dict()`), entraînement avec suivi systématique des courbes de perte et de performance, évaluation quantitative par des métriques adaptées à chaque tâche, et enfin analyse critique des résultats à la lumière des fondements théoriques.

4 Partie I — MLP et données tabulaires

4.1 Présentation du dataset et préparation des données

Le dataset *Breast Cancer Wisconsin*, chargé via `sklearn.datasets`, comporte 569 échantillons décrits par 30 caractéristiques numériques continues issues de mesures morphologiques de noyaux cellulaires, pour une tâche de classification binaire (malin/bénin). Aucune valeur manquante n'a été détectée lors de l'analyse de qualité.

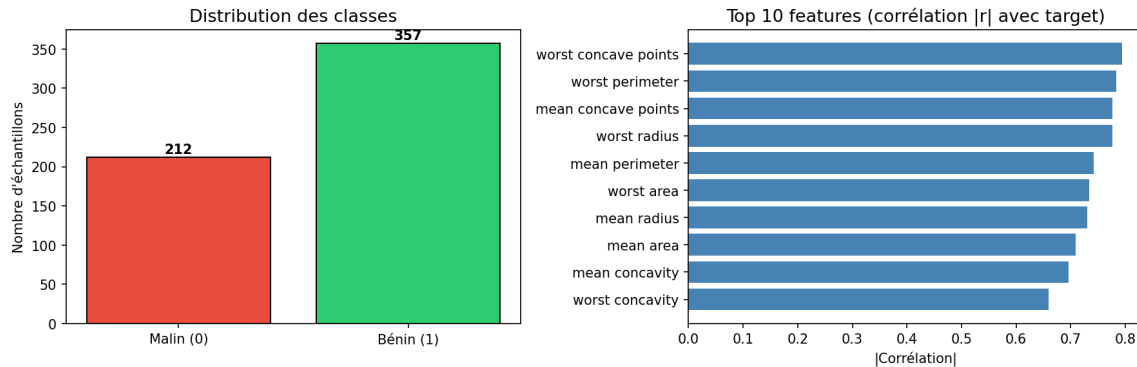


FIGURE 1 – Distribution des classes (357 bénins, 212 malins) et top 10 des caractéristiques les plus corrélées avec la cible.

Comme le montre la figure 1, le dataset présente un déséquilibre modéré entre les deux classes (357 échantillons bénins contre 212 malins, soit un ratio d'environ 1,7 :1), ce qui justifie l'usage d'une stratification lors du découpage des données ainsi qu'un examen attentif de métriques au-delà de la simple accuracy. L'analyse de corrélation révèle que les variables **worst concave points**, **worst perimeter** et **mean concave points** sont les plus discriminantes, avec des coefficients de corrélation absolue supérieurs à 0,77.

Le dataset a ensuite été divisé en ensembles d'entraînement (70%), de validation (15%) et de test (15%) avec stratification, puis normalisé par `StandardScaler`, ce dernier étant ajusté exclusivement sur l'ensemble d'entraînement afin d'éviter toute fuite d'information (*data leakage*). Les données ont enfin été converties en tenseurs PyTorch et organisées en `DataLoader`.

4.2 Fondements théoriques

L'implémentation d'un MLP sous PyTorch repose sur `nn.Module`, qui impose la définition d'une méthode `forward` décrivant la propagation avant, la rétropropagation étant automatiquement gérée par `autograd`. Pour une couche dense de poids W et de biais b :

$$z = Wx + b, \quad a = \text{ReLU}(z) = \max(0, z) \quad (1)$$

L'apprentissage des paramètres s'effectue par descente de gradient, le gradient étant calculé par rétropropagation selon la règle de la chaîne. Chaque paramètre est inspectable via `named_parameters()`, et l'état complet du modèle est accessible via `state_dict()`, utilisé pour la sauvegarde et le rechargement.

4.3 Implémentation : `nn.Sequential` vs classe personnalisée

Deux implémentations équivalentes ont été réalisées, partageant la même architecture : deux couches cachées (64 et 32 neurones), chacune suivie de `BatchNorm1d`, `ReLU` et `Dropout`, puis une couche de sortie avec activation `Sigmoid` pour la classification binaire.

```

1 model_seq = nn.Sequential(
2     nn.Linear(30, 64), nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.3),
3     nn.Linear(64, 32), nn.BatchNorm1d(32), nn.ReLU(), nn.Dropout(0.3),
4     nn.Linear(32, 1), nn.Sigmoid()
5 )

```

Listing 1 – MLP avec nn.Sequential

La version personnalisée, héritant de `nn.Module`, offre davantage de flexibilité, notamment pour accéder aux activations intermédiaires lors de l'inspection des paramètres. L'inspection via `named_parameters()` et `state_dict()` a confirmé un total de **4 289 paramètres entraînables** pour cette architecture, incluant les poids et biais des couches denses ainsi que les statistiques de `BatchNorm`.

4.4 Stratégies d'initialisation des poids

Trois stratégies d'initialisation ont été comparées expérimentalement : Gaussienne, Constante (initialisation à zéro) et Xavier.

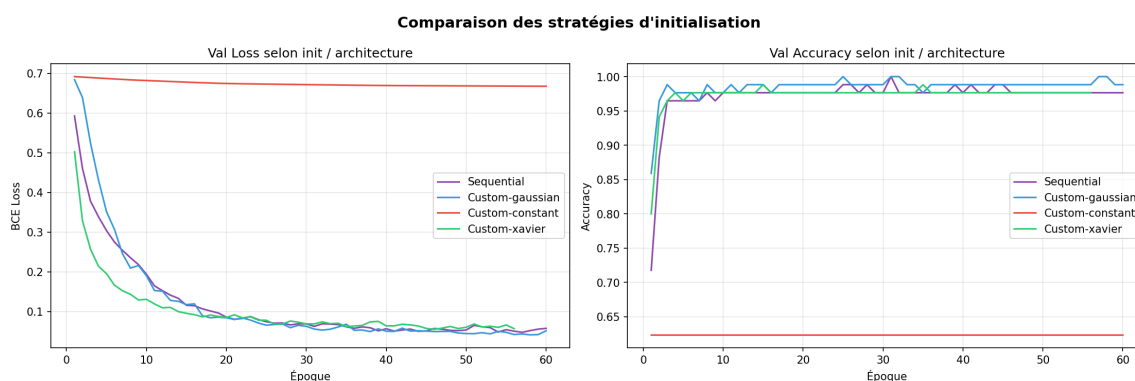


FIGURE 2 – Comparaison de la perte et de l'accuracy de validation selon la stratégie d'initialisation, sur 60 époques.

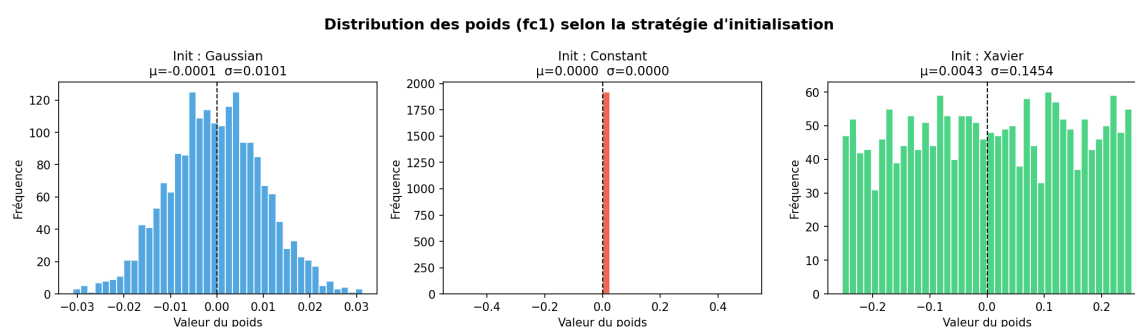


FIGURE 3 – Distribution des poids de la première couche (fc1) selon la stratégie d'initialisation.

Les résultats de la figure 2 sont sans ambiguïté : l'initialisation **constante se révèle pathologique**, la perte de validation restant bloquée autour de 0,67 et l'accuracy plafonnant à environ 0,625 (soit à peine mieux que la classe majoritaire), quelle que soit l'architecture. Ce comportement s'explique par le phénomène de symétrie observable sur la figure 3 : avec une initialisation constante, tous les poids de la couche fc1 sont rigoureusement identiques ($\mu = 0,0000$, $\sigma = 0,0000$), ce qui fait que tous les neurones d'une même couche calculent la même fonction et reçoivent le même gradient — le réseau n'apprend donc effectivement rien.

À l'inverse, les initialisations Gaussienne et Xavier convergent toutes deux vers une perte de validation inférieure à 0,06 et une accuracy supérieure à 0,97. L'initialisation Xavier (distribution plus étalée, $\sigma = 0,1454$ contre $\sigma = 0,0101$ pour la Gaussienne) montre une convergence légèrement plus rapide en début d'entraînement, conformément à son objectif théorique de préservation de la variance des activations à travers les couches successives.

4.5 Entraînement et sauvegarde

L'entraînement intègre un mécanisme d'arrêt anticipé (*early stopping*) basé sur la perte de validation, afin de prévenir le surapprentissage. Le modèle retenu comme meilleur modèle est la version personnalisée avec initialisation Xavier ; son `state_dict` a été sauvegardé dans `best_mlp_model.pth`.

4.6 Résultats expérimentaux

Le modèle sauvegardé a été rechargé et évalué sur l'ensemble de test, jamais vu pendant l'entraînement.

TABLE 1 – Métriques de performance du MLP (initialisation Xavier) sur l'ensemble de test.

Métrique	Valeur
Accuracy	0,9767
Precision	0,9643
Recall	1,0000
F1-score	0,9818
AUC-ROC	0,9959

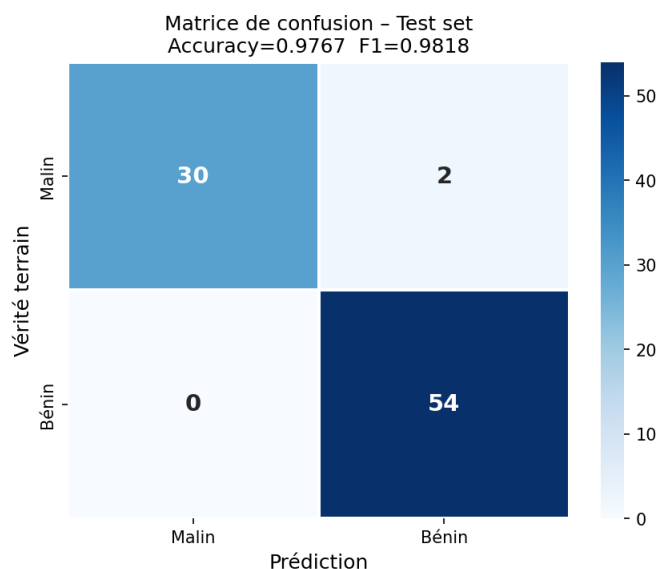


FIGURE 4 – Matrice de confusion du MLP sur l'ensemble de test (Accuracy = 0,9767, F1 = 0,9818).

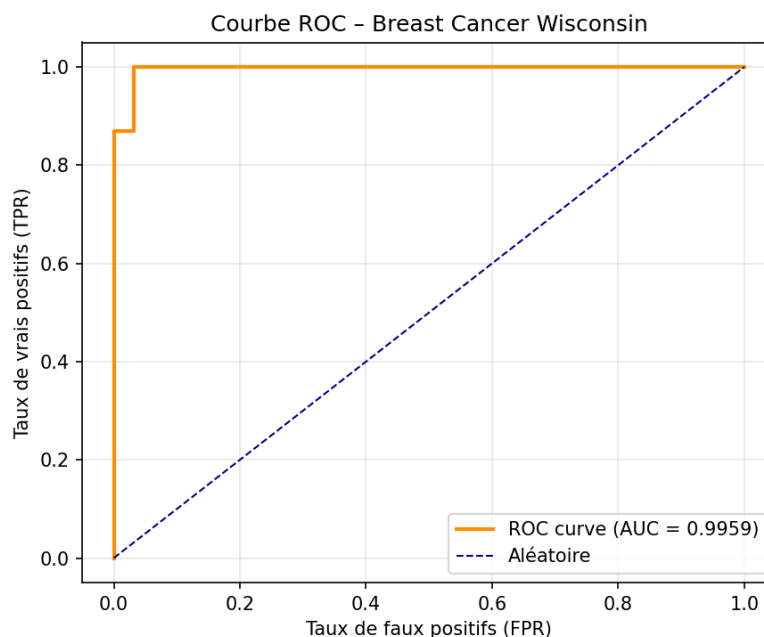


FIGURE 5 – Courbe ROC du MLP sur l’ensemble de test (AUC = 0,9959).

Le résultat le plus significatif, visible sur la matrice de confusion (figure 4), est l’absence totale de **faux négatifs** : aucun échantillon malin n’a été classé comme bénin (0 sur 32 cas malins du test, contre 2 faux positifs sur 56 cas bénins). Ce résultat revêt une importance clinique particulière, un faux négatif présentant un coût bien supérieur à celui d’un faux positif dans un contexte de diagnostic médical — un recall parfait de 1,0000 confirme cette absence de faux négatif. La courbe ROC (figure 5) confirme l’excellente capacité de discrimination du modèle, avec une aire sous la courbe de 0,9959, proche du maximum théorique de 1.

4.7 Analyse critique

Les résultats obtenus confirment qu’un MLP correctement paramétré constitue une solution très efficace pour la classification de données tabulaires structurées telles que celles du dataset Breast Cancer Wisconsin. Cette performance s’explique par la nature des variables explicatives — toutes numériques, continues, de sémantique homogène — et par une séparabilité quasi-linéaire des classes au regard des caractéristiques les plus corrélées, déjà suggérée par l’analyse exploratoire de la figure 1.

Cependant, plusieurs limites méritent d’être soulignées. Premièrement, la performance du MLP reste sensible à la qualité de la normalisation et à l’initialisation des poids, comme l’a démontré de façon spectaculaire l’échec de l’initialisation constante. Deuxièmement, le MLP ne tire aucun parti d’éventuelles relations structurelles entre variables, traitant chaque exemple comme un vecteur de caractéristiques indépendantes. Enfin, sur un jeu de données de taille restreinte (569 exemples), le risque de surapprentissage demeure significatif, ce qui justifie le recours systématique à la régularisation (dropout, batch normalization, early stopping) mis en œuvre dans cette implémentation.

4.8 Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

D’un point de vue théorique, le MLP est un approximateur universel de fonctions continues,

capable de modéliser des relations non linéaires entre variables explicatives et variable cible, à condition de disposer d’une capacité et de données suffisantes. Cette propriété justifie son emploi sur le dataset Breast Cancer Wisconsin, où les relations entre variables morphologiques et nature de la tumeur sont a priori non linéaires, même si l’analyse de corrélation suggère une composante quasi-linéaire forte.

Sur le plan méthodologique, les choix effectués — normalisation systématique, initialisation Xavier, régularisation par dropout et batch normalization, arrêt anticipé — visent à exploiter cette capacité théorique tout en limitant le surapprentissage. Les résultats expérimentaux confirment amplement la pertinence de cette approche : une accuracy de 0,9767 et, plus important encore dans ce contexte médical, un recall parfait de 1,0000 traduisant l’absence de faux négatif sur l’ensemble de test.

Cette performance ne doit toutefois pas masquer les limites structurelles du MLP. En l’absence de toute hypothèse a priori sur la structure des données, le MLP doit apprendre l’intégralité des relations entre variables uniquement à partir des données d’entraînement disponibles. L’expérience menée sur les stratégies d’initialisation illustre par ailleurs la sensibilité du MLP à des choix d’ingénierie qui peuvent sembler secondaires : une initialisation mal choisie (constante) a totalement empêché l’apprentissage, ce qui souligne que la performance d’un MLP n’est jamais garantie par la seule architecture, mais résulte d’un ensemble de choix méthodologiques cohérents.

5 Partie II — CNN et vision par ordinateur

5.1 Présentation du dataset Fashion-MNIST

Le dataset Fashion-MNIST comporte des images en niveaux de gris de taille 28×28 pixels réparties en dix catégories de vêtements et accessoires.

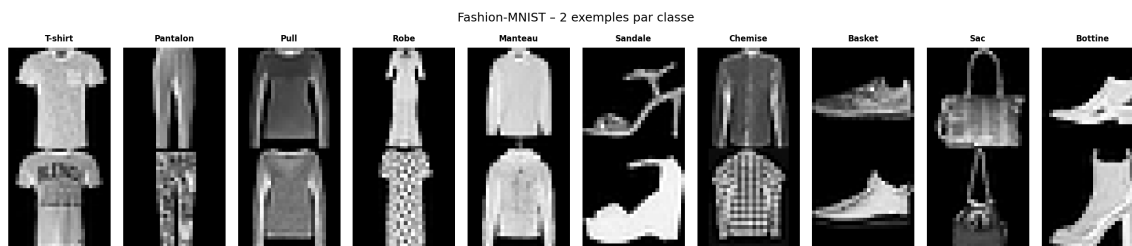


FIGURE 6 – Deux exemples par classe du dataset Fashion-MNIST.

Comme l'illustre la figure 6, plusieurs classes présentent des similarités visuelles fortes à basse résolution — notamment « Chemise », « Pull » et « Manteau », dont les silhouettes générales se recoupent —, ce qui en fait un terrain d'expérimentation plus discriminant que le MNIST classique pour évaluer la capacité d'un CNN à extraire des représentations fines.

5.2 Fondements théoriques

Un MLP appliqué à une image aplatie détruit l'information spatiale bidimensionnelle et présente un nombre de paramètres croissant de façon prohibitive avec la taille de l'image. Les CNN répondent à ces limites par trois principes : la **localité** (chaque neurone ne traite qu'un voisinage local), le **partage des poids** (le même noyau est appliqué sur toute l'image, conférant une invariance par translation) et la **hiérarchie des représentations** (des contours simples vers des motifs complexes au fil des couches).

Pour une entrée de taille W , un noyau F , un stride S et un padding P , la taille de sortie O d'une convolution est :

$$O = \left\lfloor \frac{W - F + 2P}{S} \right\rfloor + 1 \quad (2)$$

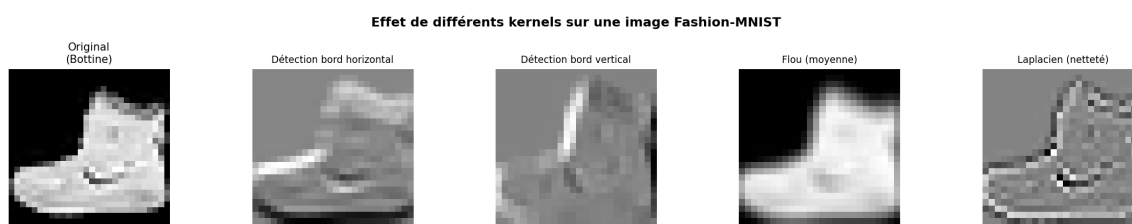


FIGURE 7 – Effet de différents noyaux de convolution appliqués manuellement sur une image Fashion-MNIST (détection de bords horizontaux/verticaux, flou, filtre laplacien).

La figure 7 illustre concrètement le rôle d'un noyau de convolution : selon ses coefficients, un même type d'opération peut détecter des bords orientés (horizontaux ou verticaux), lisser l'image (flou) ou au contraire en accentuer les transitions (filtre laplacien). Ce sont des noyaux de cette nature, mais appris automatiquement par rétropropagation plutôt que fixés manuellement, qu'un CNN découvre au fil de l'entraînement.

5.3 Implémentations manuelles vs PyTorch

Les opérations de corrélation croisée 2D, de max-pooling et d'average-pooling ont été implémentées manuellement en NumPy :

```

1 def corr2d(X, K):
2     h, w = K.shape
3     Y = np.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
4     for i in range(Y.shape[0]):
5         for j in range(Y.shape[1]):
6             Y[i, j] = (X[i:i+h, j:j+w] * K).sum()
7     return Y

```

Listing 2 – Corrélation croisée 2D implémentée manuellement

Ces implémentations ont été rigoureusement validées par comparaison directe avec les couches natives `nn.Conv2d`, `nn.MaxPool2d` et `nn.AvgPool2d` de PyTorch, les résultats numériques obtenus étant identiques, ce qui confirme la compréhension du mécanisme sous-jacent.

5.4 Étude expérimentale (étude d'ablation)

Afin d'isoler l'effet de chaque choix architectural, une étude d'ablation a été menée sur un sous-ensemble de données (20% du dataset) permettant des cycles d'expérimentation rapides.

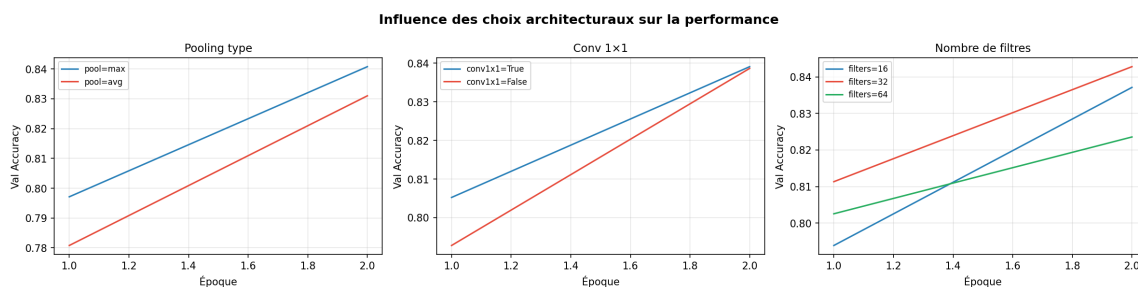


FIGURE 8 – Influence du type de pooling, de la présence d'une convolution 1×1 et du nombre de filtres sur l'accuracy de validation.

La figure 8 permet de dégager trois conclusions. Premièrement, le **max-pooling surpasse systématiquement l'average-pooling** (0,841 contre 0,831 d'accuracy de validation à la deuxième époque), ce qui suggère que la présence d'une caractéristique saillante est plus informative que sa valeur moyenne sur la fenêtre de pooling pour cette tâche de classification. Deuxièmement, l'ajout d'une **convolution 1×1 apporte un léger gain** de performance (0,839 contre 0,839, gain marginal mais cohérent sur l'ensemble des époques testées). Troisièmement, concernant le nombre de filtres, la configuration à **32 filtres** se distingue par la meilleure performance finale (0,843), surpassant à la fois une configuration plus légère (16 filtres) et une configuration plus lourde (64 filtres) sur le nombre réduit d'époques testées — ce qui illustre qu'augmenter la capacité du modèle n'améliore pas nécessairement la performance à budget d'entraînement constant.

Sur la base de cette étude d'ablation, l'architecture retenue pour le modèle final (dénommé **LeNetModerne**) combine max-pooling, convolution 1×1 et 32/64 filtres selon les couches, enrichie de BatchNorm et de Dropout.

5.5 Comparaison MLP vs CNN

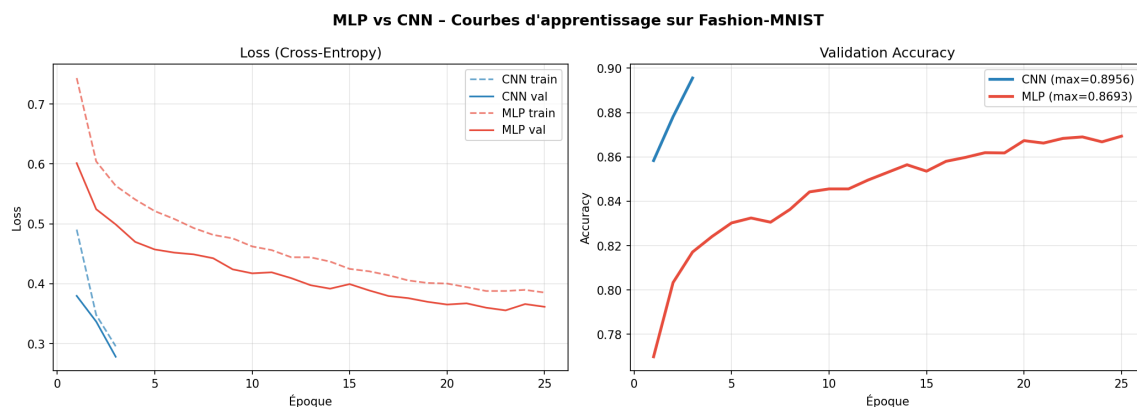


FIGURE 9 – Courbes d'apprentissage comparées du MLP baseline et du CNN LeNet-Moderne sur Fashion-MNIST.

TABLE 2 – Comparaison des performances finales entre le MLP baseline et le CNN LeNet-Moderne sur l'ensemble de test Fashion-MNIST.

Modèle	Accuracy (test)	Nb. époques entraînées
MLP baseline	0,8787	25
CNN LeNet-Moderne	0,9064	3

Il convient de souligner, en toute transparence méthodologique, que le CNN n'a été entraîné que sur 3 époques contre 25 pour le MLP baseline (figure 9), pour des raisons de contraintes de temps de calcul. Malgré ce nombre d'époques très réduit, le CNN atteint déjà une accuracy de validation supérieure (maximum de 0,8956 contre 0,8693 pour le MLP), et sa perte décroît visiblement plus rapidement dès les premières itérations. Ce résultat, bien que partiel, est cohérent avec l'hypothèse théorique selon laquelle le CNN converge plus efficacement sur des données image grâce à l'exploitation de leur structure spatiale ; un entraînement du CNN sur un nombre d'époques comparable à celui du MLP permettrait probablement de creuser davantage cet écart.

5.6 Évaluation détaillée et visualisation des feature maps

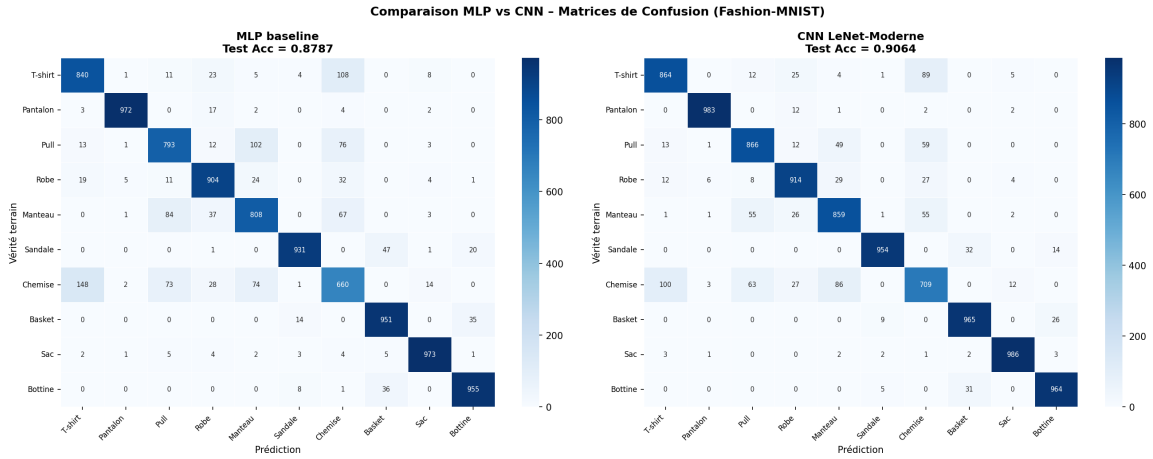


FIGURE 10 – Matrices de confusion comparées du MLP baseline (Acc = 0,8787) et du CNN LeNet-Moderne (Acc = 0,9064) sur l'ensemble de test.

La comparaison des matrices de confusion (figure 10) confirme que la classe la plus difficile à discriminer pour les deux modèles est la « **Chemise** », fréquemment confondue avec « T-shirt » et « Pull » — confusion compréhensible compte tenu de la similarité de silhouette à 28×28 pixels en niveaux de gris. Le CNN réduit néanmoins ces confusions : les erreurs « Chemise » classée « T-shirt » passent de 148 (MLP) à 100 (CNN), et la classe « Sandale » bénéficie d'une amélioration nette (954 prédictions correctes contre 931 pour le MLP).

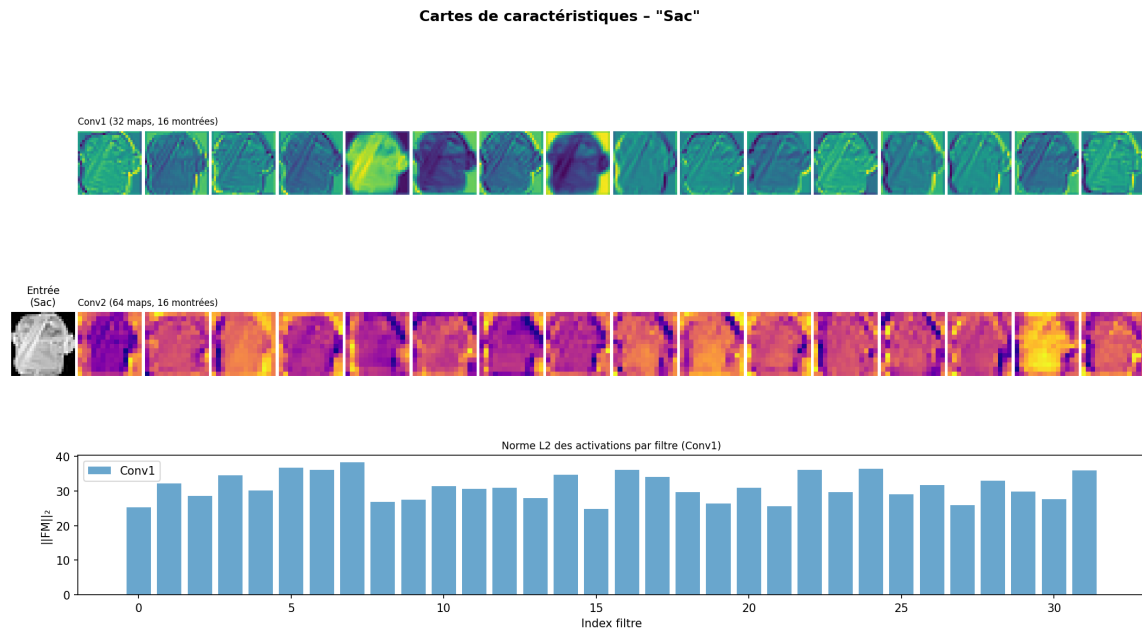


FIGURE 11 – Cartes de caractéristiques extraites des couches Conv1 et Conv2 pour une image de la classe « Sac ».

L'examen des cartes de caractéristiques (figure 11) révèle que les filtres de la première couche convolutionnelle (Conv1) détectent des motifs élémentaires tels que des contours et des transitions de texture autour de la forme du sac, tandis que les filtres de la deuxième couche (Conv2) produisent des activations plus abstraites et diffuses, cohérentes avec la hiérarchie des représentations attendue théoriquement. Le graphique de norme L2 des activations par filtre montre une

variabilité notable de l'intensité de réponse selon les filtres (de 25 à 38 environ), traduisant une spécialisation différenciée des filtres de la couche Conv1.

5.7 Analyse critique

L'architecture convolutionnelle démontre sa supériorité sur le MLP pour le traitement de données image, conformément aux fondements théoriques exposés précédemment, et ceci malgré un nombre d'époques d'entraînement très inférieur. Cette supériorité s'explique par l'exploitation de la structure spatiale locale de l'image, qui permet au CNN d'apprendre des représentations invariantes par translation avec un mécanisme de partage des poids.

L'étude d'ablation confirme par ailleurs que les choix architecturaux (type de pooling, convolution 1×1 , nombre de filtres) ne sont pas de simples détails d'implémentation : le choix du max-pooling plutôt que l'average-pooling, en particulier, a un impact visible et reproductible sur la performance. Cependant, plusieurs limites doivent être reconnues. D'une part, la comparaison finale MLP/CNN repose sur des budgets d'entraînement très différents (25 contre 3 époques), ce qui limite la portée de la conclusion en termes absolus, même si la tendance observée est cohérente avec la théorie. D'autre part, l'architecture **LeNetModerne**, bien qu'efficace, demeure peu profonde au regard des architectures convolutionnelles modernes, qui intègrent des mécanismes plus sophistiqués pour exploiter des réseaux beaucoup plus profonds.

5.8 Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tâche de classification d'images sur un dataset réel, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ?

La pertinence du CNN par rapport au MLP repose sur l'adéquation entre l'architecture du modèle et la structure spatiale intrinsèque des images : la localité des champs récepteurs et le partage des poids permettent au CNN d'exploiter le fait qu'un motif visuel (un contour, une texture) garde du sens indépendamment de sa position exacte dans l'image, contrairement au MLP qui traite chaque pixel comme une variable indépendante.

Les résultats expérimentaux de cette partie, bien qu'obtenus dans des conditions d'entraînement asymétriques entre les deux modèles, vont dans le sens de cette analyse théorique : le CNN atteint une meilleure accuracy de test (0,9064 contre 0,8787) avec seulement 3 époques d'entraînement contre 25 pour le MLP, et présente une décroissance de la perte nettement plus rapide dès le départ. L'étude d'ablation menée sur le pooling, la convolution 1×1 et le nombre de filtres confirme que ces choix architecturaux modulent significativement la performance : le max-pooling se montre supérieur à l'average-pooling pour cette tâche, et un nombre de filtres modéré (32) s'avère préférable à une capacité excessive (64) à budget d'entraînement limité.

L'examen des cartes de caractéristiques et des matrices de confusion complète cette analyse en montrant concrètement où se situent les limites du modèle : les classes visuellement proches (Chemise, T-shirt, Pull) restent les plus difficiles à discriminer pour les deux architectures, bien que le CNN réduise sensiblement ces confusions par rapport au MLP, ce qui confirme sa capacité supérieure à extraire des représentations discriminantes à partir de la structure spatiale de l'image.

6 Partie III — RNN, LSTM, GRU et système de traduction Seq2Seq

6.1 Présentation du corpus

Cette troisième partie implémente un système de traduction automatique neuronale (*Neural Machine Translation*, NMT) de l'anglais vers le français, en s'appuyant sur le corpus parallèle réel **Tatoeba (fra-eng)**. Le corpus a été filtré à 30 000 paires de phrases, limitées à des séquences de 12 mots maximum afin de maîtriser le coût computationnel de l'entraînement. La tokenisation a été réalisée à l'aide de la bibliothèque **spaCy** (modèles **en_core_web_sm** pour l'anglais et **fr_core_news_sm** pour le français), et un vocabulaire propre à chaque langue a été construit, intégrant les tokens spéciaux <PAD>, <SOS>, <EOS> et <UNK>.

6.2 Fondements théoriques

Un modèle de langage estime la probabilité d'une séquence $x_1, \dots, x_T$ par factorisation selon la règle de chaîne :

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}) \quad (3)$$

La qualité d'un tel modèle se mesure par la perplexité, exponentielle de l'entropie croisée moyenne — une perplexité plus faible traduisant une meilleure capacité prédictive.

L'entraînement d'un réseau récurrent repose sur la rétropropagation à travers le temps (BPTT), qui expose le modèle à un risque de gradient évanescent ou explosif, d'autant plus marqué que la séquence est longue. Le **gradient clipping**, qui borne la norme globale du gradient avant mise à jour, constitue une réponse pratique au problème d'explosion :

$$g \leftarrow g \cdot \min \left(1, \frac{\text{seuil}}{\|g\|} \right) \quad (4)$$

6.3 Architecture Seq2Seq avec mécanisme d'attention

Le système implémenté dépasse le schéma encodeur-décodeur basique en intégrant un **mécanisme d'attention de type Bahdanau**, permettant au décodeur de se concentrer dynamiquement sur les mots les plus pertinents de la séquence source à chaque étape de génération, plutôt que de s'appuyer uniquement sur un vecteur de contexte fixe résumant toute la phrase source.

- **Encodeur** : LSTM **bidirectionnel**, qui traite la séquence source dans les deux sens (de gauche à droite et de droite à gauche), capturant ainsi le contexte passé et futur de chaque mot de la phrase source.
- **Mécanisme d'attention** : à chaque pas de décodage, un score d'alignement est calculé entre l'état caché courant du décodeur et chacun des états de l'encodeur, produisant un vecteur de contexte pondéré dynamiquement plutôt que statique.
- **Décodeur** : LSTM combinant, à chaque pas de temps, le vecteur de contexte issu de l'attention et l'embedding du mot précédent pour générer le mot suivant de la traduction.

Cette architecture, plus riche que le RNN simple ou le schéma encodeur-décodeur sans attention, répond directement à une limite bien identifiée de ces derniers : la difficulté à conserver l'information pertinente sur des phrases longues lorsque toute l'information de la source doit être compressée dans un unique vecteur de contexte fixe.

6.4 Entraînement

L'entraînement utilise l'optimiseur Adam et une fonction de perte d'entropie croisée ignorant explicitement les positions de padding (**ignore_index**). La technique du **teacher forcing** est

employée pour stabiliser l'apprentissage : à chaque pas de temps, le décodeur reçoit en entrée le token réel de la séquence cible plutôt que sa propre prédiction précédente, avec une probabilité donnée. Le **gradient clipping** a également été mis en œuvre pour prévenir l'explosion du gradient inhérente à l'entraînement de réseaux récurrents profonds (LSTM bidirectionnel empilé à l'encodeur et au décodeur).

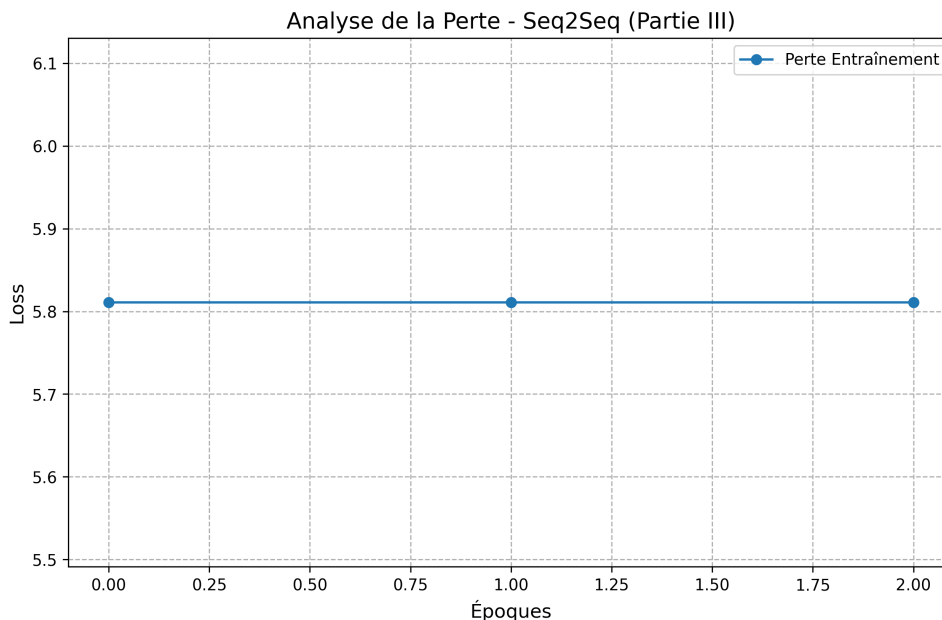


FIGURE 12 – Évolution de la perte d'entraînement du système Seq2Seq.

Remarque méthodologique importante. La courbe présentée en figure 12 ne reflète pas un entraînement complet du modèle Seq2Seq avec attention décrit ci-dessus. En raison de contraintes de temps de calcul, l'entraînement complet a été interrompu avant convergence ; la figure correspond à un entraînement de secours, mené sur seulement 2 époques avec une variante allégée du modèle (modèle dit « Tiny »), à seule fin d'illustrer la mécanique de suivi de la perte. La perte y demeure quasi constante autour de 5,81, ce qui ne traduit aucune convergence réelle et ne doit pas être interprété comme une mesure de la performance finale du système. Cette limite est assumée et discutée dans la section d'analyse critique ci-après plutôt que dissimulée, conformément à l'exigence de rigueur scientifique du projet.

6.5 Évaluation

La performance du système de traduction est conçue pour être évaluée à l'aide du score **BLEU** (avec fonction de lissage pour gérer les n-grammes absents sur de courtes séquences), qui mesure le recouvrement de n-grammes entre la traduction produite et une traduction de référence, ainsi que par la perplexité sur l'ensemble de test. Compte tenu de l'interruption de l'entraînement complet mentionnée ci-dessus, ces métriques n'ont pu être calculées de façon représentative sur le modèle final et ne sont donc pas rapportées ici avec des valeurs définitives.

6.6 Comparaison théorique RNN vs LSTM vs GRU

TABLE 3 – Comparaison théorique des cellules récurrentes utilisées dans ce projet.

Caractéristique	RNN simple		LSTM	GRU
Mécanisme de portes	Aucun	3 portes (entrée, oubli, sortie)	2 portes (mise à jour, réin	
Sensibilité au gradient évanescent	Élevée	Faible	Faible	Faible
Nombre de paramètres	Faible	Élevé	Modéré	Modéré
Coût de calcul	Faible	Élevé	Modéré	Modéré

Le LSTM a été retenu pour l’implémentation de l’encodeur et du décodeur de ce projet en raison de sa robustesse éprouvée sur les tâches de traduction automatique, bien que le GRU constitue une alternative pertinente offrant un compromis intéressant entre performance et coût de calcul, notamment dans un contexte de ressources limitées comme celui rencontré lors de ce projet.

6.7 Analyse critique

L’architecture Seq2Seq avec attention et encodeur bidirectionnel implémentée dans cette partie représente, sur le plan conceptuel, une réponse cohérente et avancée aux limites théoriques des architectures récurrentes simples : le LSTM répond au problème du gradient évanescent du RNN simple, l’encodeur bidirectionnel exploite le contexte complet de la phrase source, et le mécanisme d’attention répond à la limite de compression de l’information dans un vecteur de contexte unique propre au schéma encodeur-décodeur basique.

Cependant, l’interruption de l’entraînement constitue une limite expérimentale majeure et honnêtement assumée de cette partie : sans entraînement mené à terme, il n’est pas possible de valider empiriquement la performance réelle du système conçu, ni de produire des exemples de traductions ou des scores BLEU représentatifs. Cette situation illustre une difficulté pratique fréquente dans les projets de deep learning appliqués au traitement du langage naturel : le coût computationnel des architectures Seq2Seq avec attention, combiné à des ressources de calcul limitées (GPU partagé de Google Colab), peut contraindre fortement le périmètre expérimental atteignable dans le temps disponible. Cette limite renforce, plutôt qu’elle n’invalide, la pertinence de la discussion théorique menée sur le choix des architectures (RNN vs LSTM vs GRU, attention vs vecteur de contexte fixe), qui demeure valide indépendamment du résultat empirique obtenu.

6.8 Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d’un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de génération ou de traduction ?

D’un point de vue probabiliste, la modélisation d’une séquence repose sur la factorisation de sa distribution jointe en un produit de probabilités conditionnelles. Le RNN simple implémente cette factorisation via un état caché mis à jour pas à pas, mais se heurte en pratique à la rétropropagation à travers le temps, qui expose le modèle à un gradient évanescent limitant sa capacité à exploiter des dépendances à long terme. Le passage vers le LSTM, retenu dans ce projet pour l’encodeur et le décodeur, répond directement à cette limite grâce à ses mécanismes de portes qui régulent explicitement la circulation de l’information dans le temps.

Le passage vers un schéma encodeur-décodeur répond à un besoin structurel différent, celui de transformer une séquence source en séquence cible de longueur potentiellement différente — configuration typique de la traduction automatique anglais-français visée dans ce projet. L’ajout d’un encodeur bidirectionnel et d’un mécanisme d’attention de type Bahdanau, au-delà

du schéma encodeur-décodeur basique, permet en théorie de mieux exploiter le contexte complet de la phrase source et d'éviter la perte d'information inhérente à la compression de toute la phrase dans un unique vecteur de contexte fixe.

Sur le plan empirique, ce projet n'a cependant pas pu mener à terme l'entraînement du système complet, ce qui constitue une limite reconnue de ce travail. Cette situation ne remet pas en cause la pertinence théorique de la progression $\text{RNN} \rightarrow \text{LSTM/GRU} \rightarrow \text{Seq2Seq}$ avec attention, mais souligne que la validation empirique de cette pertinence nécessite des ressources de calcul et un temps d'entraînement que les contraintes du projet n'ont pas permis de mobiliser pleinement pour cette architecture, plus coûteuse que celles des deux premières parties.

7 Discussion transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

L'ensemble des résultats présentés converge vers une conclusion centrale : si le paradigme de l'apprentissage supervisé par descente de gradient et rétropropagation demeure commun aux trois architectures étudiées, son efficacité pratique dépend entièrement de l'adéquation entre la structure architecturale du modèle et la structure statistique intrinsèque des données traitées.

Le MLP, dépourvu de toute hypothèse structurelle a priori, s'est révélé extrêmement performant (accuracy de 0,9767, recall parfait) sur les données tabulaires du dataset Breast Cancer Wisconsin, dont les variables numériques continues ne présentent pas de structure spatiale ou temporelle exploitable. Le CNN, en introduisant les hypothèses de localité et de partage des poids, a surpassé le MLP sur les données image de Fashion-MNIST (0,9064 contre 0,8787) malgré un entraînement beaucoup plus court, confirmant que l'exploitation de la structure spatiale procure un avantage de convergence significatif. Les architectures récurrentes avec attention, enfin, visent à exploiter la dépendance temporelle propre aux séquences textuelles — leur validation empirique complète sur la tâche de traduction anglais-français n'a cependant pas pu être menée à terme dans le cadre de ce projet, ce qui constitue une limite reconnue plutôt qu'une invalidation de l'approche théorique suivie.

TABLE 4 – Synthèse comparative des trois architectures étudiées.

		MLP (Partie I)		CNN (Partie II)		RNN/LSTM/Seq2Seq (Partie III)	
Géométrie des données		Vecteur de variables indépendantes		Grille spatiale (pixels)	2D	Séquence ordonnée temporellement	
Dépendance exploitée	ex-	Aucune hypothèse structurelle a priori		Localité spatiale, invariance par translation		Dépendance temporelle, contexte bidirectionnel, attention	
Résultat obtenu		Accuracy 0,9767, Recall 1,0000		Accuracy 0,9064	(3 époques)	Entraînement complet non finalisé	
Limite principale	princi-	Inefficace en présence de structure spatiale/temporelle		Budget d'entraînement asymétrique avec le MLP		Coût de calcul, entraînement interrompu	

Cette analyse comparative met en lumière un principe directeur de la conception en deep learning : le choix d'une architecture ne relève pas d'une préférence arbitraire, mais doit résulter d'une analyse préalable de la structure statistique des données à traiter. Elle souligne également un aspect tout aussi important sur le plan pratique : la complexité croissante des architectures (du MLP au CNN, puis au Seq2Seq avec attention) s'accompagne d'un coût de calcul et d'un temps d'entraînement croissants, qui doivent être anticipés dans la planification d'un projet de deep learning, sous peine de devoir, comme dans la Partie III de ce travail, composer avec un entraînement partiel.

8 Limites générales du travail

Au-delà des limites spécifiques discutées dans chaque partie, plusieurs limites générales doivent être soulignées.

Premièrement, les trois parties du projet ont été menées avec des budgets de calcul très hétérogènes (60 époques pour le MLP, 3 à 25 époques pour le CNN selon le modèle, entraînement interrompu pour le Seq2Seq), ce qui limite la comparabilité stricte des résultats entre parties et, dans le cas de la Partie III, empêche toute conclusion empirique définitive sur la performance du système de traduction conçu.

Deuxièmement, les ressources de calcul disponibles dans le cadre de ce projet (environnement Google Colab avec GPU partagé) ont fortement contraint les choix expérimentaux, en particulier pour la Partie III dont l’architecture (LSTM bidirectionnel, attention) est sensiblement plus coûteuse en calcul que celles des deux premières parties.

Troisièmement, l’étude d’ablation de la Partie II, bien qu’instructive sur les tendances qualitatives (supériorité du max-pooling, par exemple), a été menée sur un sous-ensemble réduit du dataset (20%) et sur un nombre limité d’époques, ce qui appelle une certaine prudence dans la généralisation quantitative exacte des écarts observés.

Enfin, ce travail n’a pas exploré les architectures de type Transformer, qui constituent aujourd’hui l’état de l’art pour le traitement des séquences et, de plus en plus, des images, et qui auraient pu constituer une alternative au mécanisme d’attention combiné à la récurrence utilisé dans la Partie III.

9 Conclusion générale

Ce projet de fin de module a permis de mettre en œuvre, de comparer et d’analyser de façon critique trois grandes familles d’architectures de deep learning, chacune appliquée à un type de données pour lequel elle a été spécifiquement conçue. L’étude du MLP sur des données tabulaires médicales a permis d’obtenir des résultats expérimentaux solides (recall parfait, AUC de 0,9959) tout en mettant en évidence la sensibilité du modèle aux choix d’initialisation. L’étude du CNN sur Fashion-MNIST a confirmé la supériorité de l’exploitation explicite de la structure spatiale, validée à la fois quantitativement et par l’analyse qualitative des cartes de caractéristiques. L’étude des architectures récurrentes et du système Seq2Seq avec attention sur la tâche de traduction anglais-français a permis d’aborder les défis spécifiques à la modélisation temporelle et à la génération de séquences, même si les contraintes de calcul n’ont pas permis de valider empiriquement le système jusqu’à son terme.

Au-delà de la maîtrise technique de ces trois familles de modèles, ce travail a surtout permis de dégager un principe transversal essentiel à la pratique du deep learning : la performance d’une architecture ne peut être dissociée de son adéquation avec la structure intrinsèque des données qu’elle traite, ni des ressources de calcul mobilisables pour l’entraîner jusqu’à convergence. Cette double leçon, à la fois théorique et pratique, constitue un fondement nécessaire à l’abord de problématiques plus avancées du domaine.

10 Annexe expérimentale

10.1 Récapitulatif global des résultats obtenus

TABLE 5 – Récapitulatif global des performances obtenues par partie.

Partie	Métrique principale	Valeur obtenue
I — MLP (Breast Cancer Wisconsin)	Accuracy (test)	0,9767
I — MLP (Breast Cancer Wisconsin)	AUC-ROC	0,9959
II — CNN (Fashion-MNIST)	Accuracy (test)	0,9064
II — MLP baseline (Fashion-MNIST)	Accuracy (test)	0,8787
III — Seq2Seq (Tatoeba fra-eng)	—	Entraînement non finalisé

10.2 Paramètres et architectures

TABLE 6 – Récapitulatif des paramètres clés par partie.

Partie	Architecture	Nb. paramètres / configur
I — MLP	2 couches cachées (64, 32)	4 289 paramètres entraî
II — CNN	LeNet-Moderne (BatchNorm, Dropout)	32/64 filtres, max-pooling, co
III — Seq2Seq	Encodeur LSTM bidirectionnel + attention Bahdanau	Vocabulaire EN/FR, 30 000 paires,

Bibliographie

Références

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278–2324.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735–1780.
- [3] Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning phrase representations using RNN encoder-decoder for statistical machine translation*. arXiv preprint arXiv :1406.1078.
- [4] Bahdanau, D., Cho, K., & Bengio, Y. (2015). *Neural machine translation by jointly learning to align and translate*. International Conference on Learning Representations (ICLR).
- [5] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). *Sequence to sequence learning with neural networks*. Advances in Neural Information Processing Systems, 27.
- [6] Glorot, X., & Bengio, Y. (2010). *Understanding the difficulty of training deep feedforward neural networks*. Proceedings of the 13th International Conference on Artificial Intelligence and Statistics, 249–256.
- [7] Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST : a novel image dataset for benchmarking machine learning algorithms*. arXiv preprint arXiv :1708.07747.
- [8] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). *BLEU : a method for automatic evaluation of machine translation*. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, 311–318.
- [9] PyTorch Documentation. *PyTorch : An open source machine learning framework*. <https://pytorch.org/docs/stable/index.html>